

DSS5201 Data Visualisation

Week 12 Visualising Geospatial Data with GeoPandas

David CHEW

Semester 2 AY25/26

Last Week: plotnine

- The `plotnine` Way
- Relational Plots
- Continuous Distributions
- Categorical Distributions
- Choropleth Maps

This Week:

- GeoPandas
- plotly

GeoPandas

Visualising Geospatial Data

`GeoPandas` is a powerful Python library for analysing and visualising geospatial data.

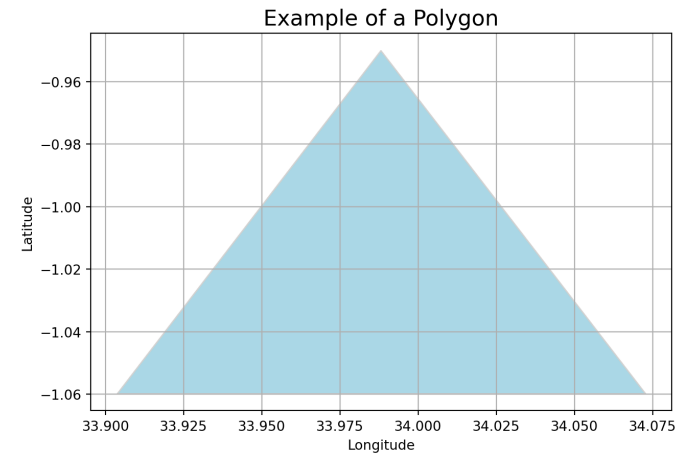
- Extends `pandas` functionalities.
- The core data structure is a `GeoDataFrame`, a subclass of `pandas' DataFrame`.



GeoDataFrame

A specialized table where the **geometry** column stores the coordinates, shapes, and spatial boundaries for every entry.

- A **point** is a single location.
- A **polygon** is a single area defined by a set of coordinates.
- A **multipolygon** is a more general geometric type that contains a collection of multiple polygons.



Import Libraries

- Let's import the necessary libraries for this session.
- We will also use a sample data frame from [geodatasets](#).

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import geopandas as gpd
4 import geodatasets
```



GeoDataFrame

We will revisit the `geoda.sids` data for a map of North Carolina.

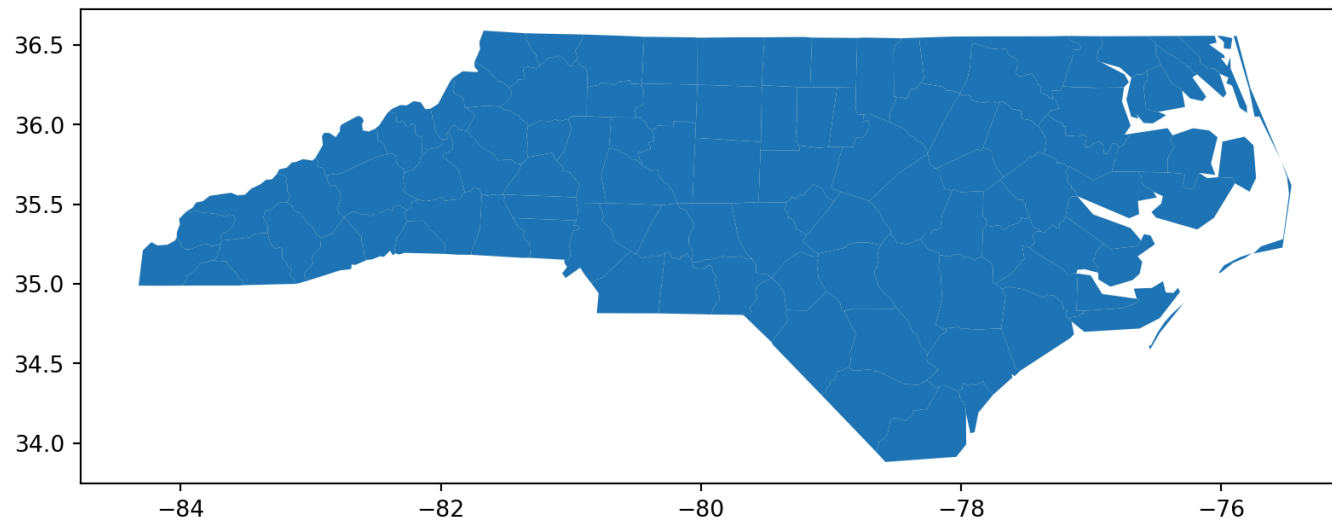
```
1 path = geodatasets.get_path('geoda.sids')
2 df = gpd.read_file(path)
3 df[['NAME', 'BIR79', 'geometry']].head()
```

	NAME	BIR79	geometry
0	Ashe	1364.0	POLYGON ((-81.47276 36.23436, -81.54084 36.272...
1	Alleghany	542.0	POLYGON ((-81.23989 36.36536, -81.24069 36.379...
2	Surry	3616.0	POLYGON ((-80.45634 36.24256, -80.47639 36.254...
3	Currituck	830.0	MULTIPOLYGON (((-76.00897 36.3196, -76.01735 3...
4	Northampton	1606.0	POLYGON ((-77.21767 36.24098, -77.23461 36.214...

SIDS Data: First Map with GeoPandas

- The `.plot()` method from `matplotlib` plots each region on a map.

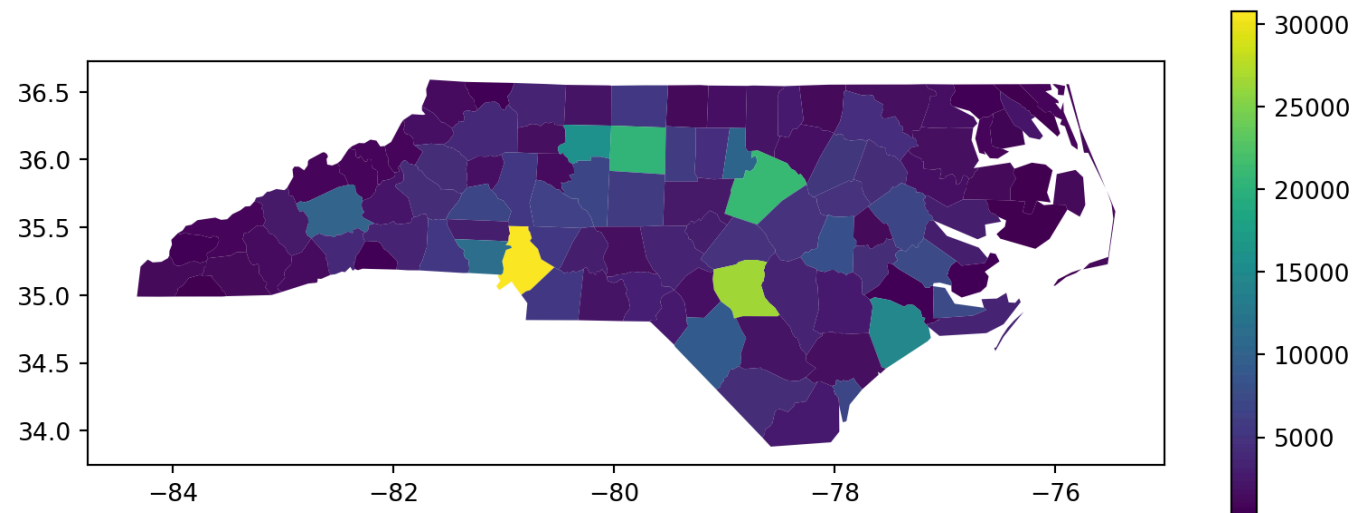
```
1 df.plot()  
2 plt.show()
```



SIDS Data: First Map with GeoPandas

- Color-code the regions according to `BIR79`:

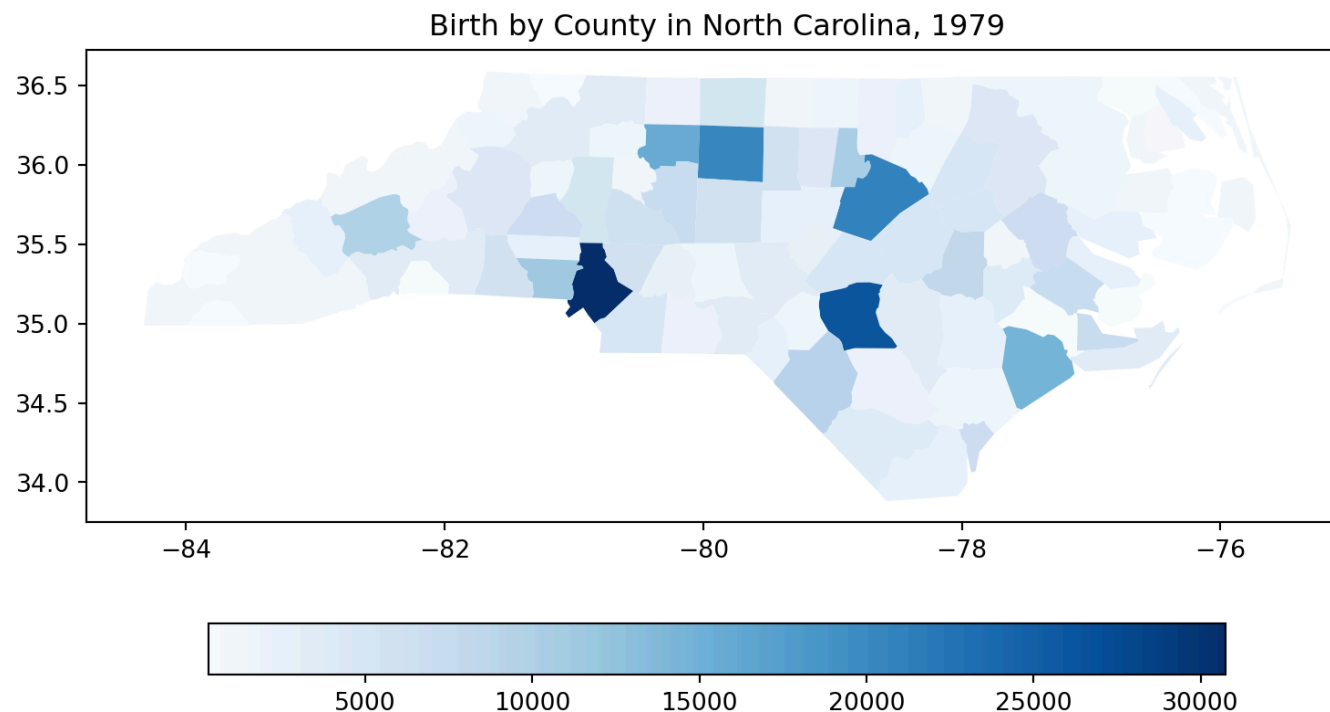
```
1 df.plot(column = 'BIR79', legend = True, legend_kwds = {'shrink': 0.75})  
2 plt.show()
```



SIDS Data: First Map with GeoPandas

Further Customisations

```
1 df.plot(column = 'BIR79', legend = True, cmap = 'Blues',  
2         legend_kws = {'orientation': 'horizontal', 'shrink': 0.75})  
3 plt.title('Birth by County in North Carolina, 1979')  
4 plt.show()
```



The World Map

Natural Earth is a public domain data source that is useful for creating maps.

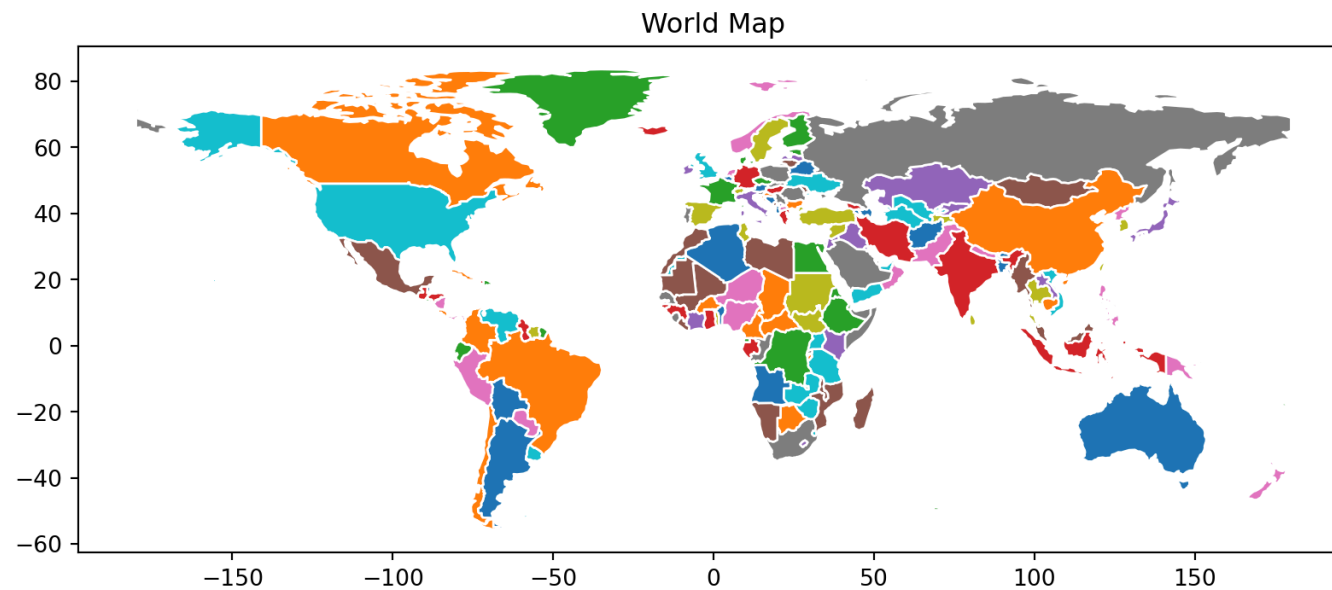
<https://www.naturalearthdata.com/downloads/110m-cultural-vectors/110m-admin-0-countries/>

```
1 path = '../data/ne_110m_admin_0_countries.zip'
2 df = gpd.read_file(path)
3 df = df[df['SOVEREIGNT'] != 'Antarctica']
4 df[['SOVEREIGNT', 'ISO_A3', 'geometry', 'POP_EST', 'GDP_MD']].head()
```

	SOVEREIGNT	ISO_A3	geometry	POP_EST	GDP_MD
0	Fiji	FJI	MULTIPOLYGON (((180 -16.06713, 180 -16.55522, ...	889953.0	5496
1	United Republic of Tanzania	TZA	POLYGON ((33.90371 -0.95, 34.07262 -1.05982, 3...	58005463.0	63177
2	Western Sahara	ESH	POLYGON ((-8.66559 27.65643, -8.66512 27.58948...	603253.0	907
3	Canada	CAN	MULTIPOLYGON (((-122.84 49, -122.97421 49.0025...	37589262.0	1736425

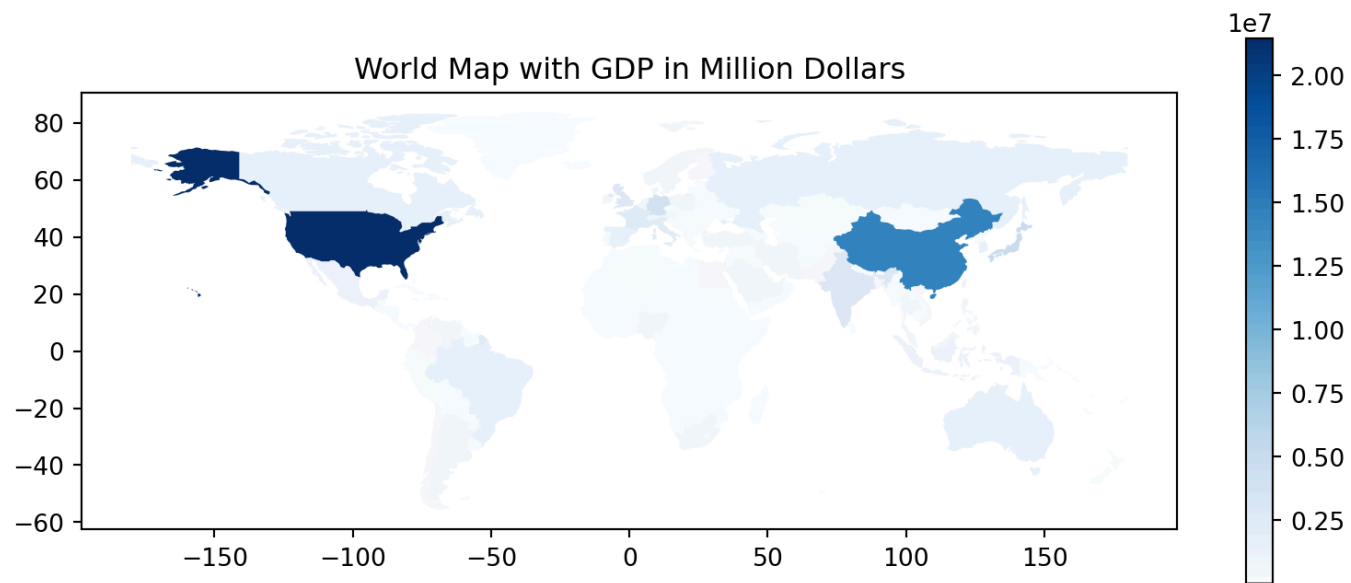
The World Map

```
1 df.plot(column = 'SOVEREIGNT', edgecolor = 'white')  
2 plt.title('World Map')  
3 plt.show()
```



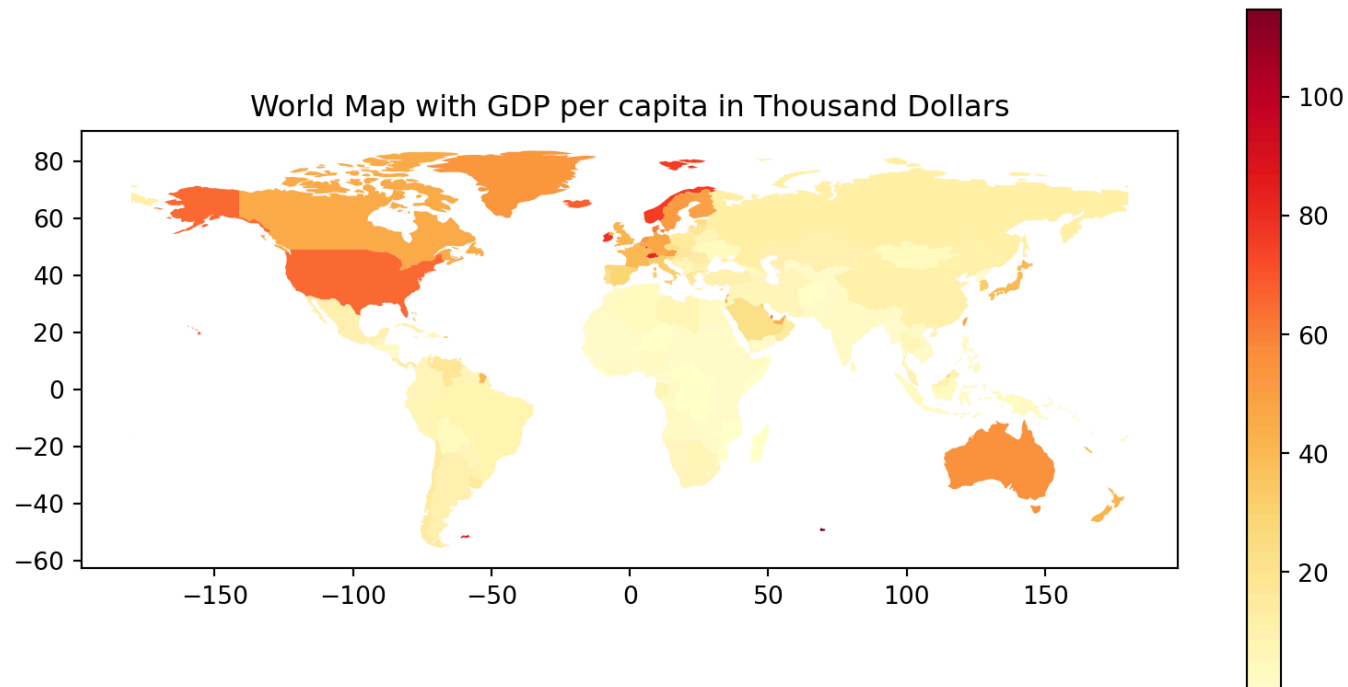
The World Map

```
1 df.plot('GDP_MD', legend = True, legend_kwds = {'shrink': 0.8}, cmap = 'Blues')
2 plt.title('World Map with GDP in Million Dollars')
3 plt.show()
```



The World Map

```
1 df['GDP_PC'] = df['GDP_MD']/df['POP_EST']*1000
2 df.plot(column = 'GDP_PC', legend = True, cmap = 'YlOrRd')
3 plt.title('World Map with GDP per capita in Thousand Dollars')
4 plt.show()
```



The World Map

Creating GeoDataFrame from DataFrame

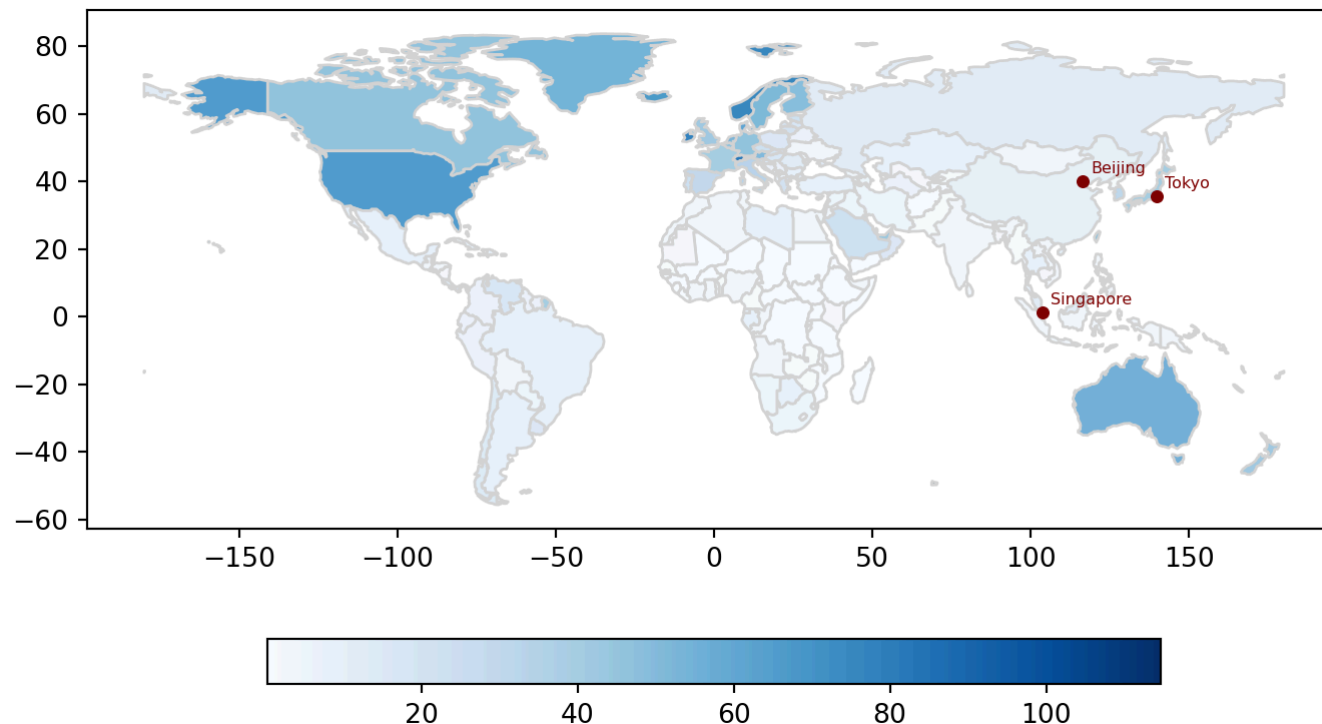
We can highlight cities on the map by geo-coding them.

```
1 cities = pd.DataFrame({
2     'City':      ['Tokyo', 'Singapore', 'Beijing'],
3     'Latitude':  [35.68,   1.35,      39.90],
4     'Longitude': [139.69,  103.82,   116.40]
5 })
6 cities = gpd.GeoDataFrame(
7     cities, crs = 'EPSG:4326',
8     geometry = gpd.points_from_xy(cities['Longitude'], cities['Latitude']))
9 cities
```

	City	Latitude	Longitude	geometry
0	Tokyo	35.68	139.69	POINT (139.69 35.68)
1	Singapore	1.35	103.82	POINT (103.82 1.35)
2	Beijing	39.90	116.40	POINT (116.4 39.9)

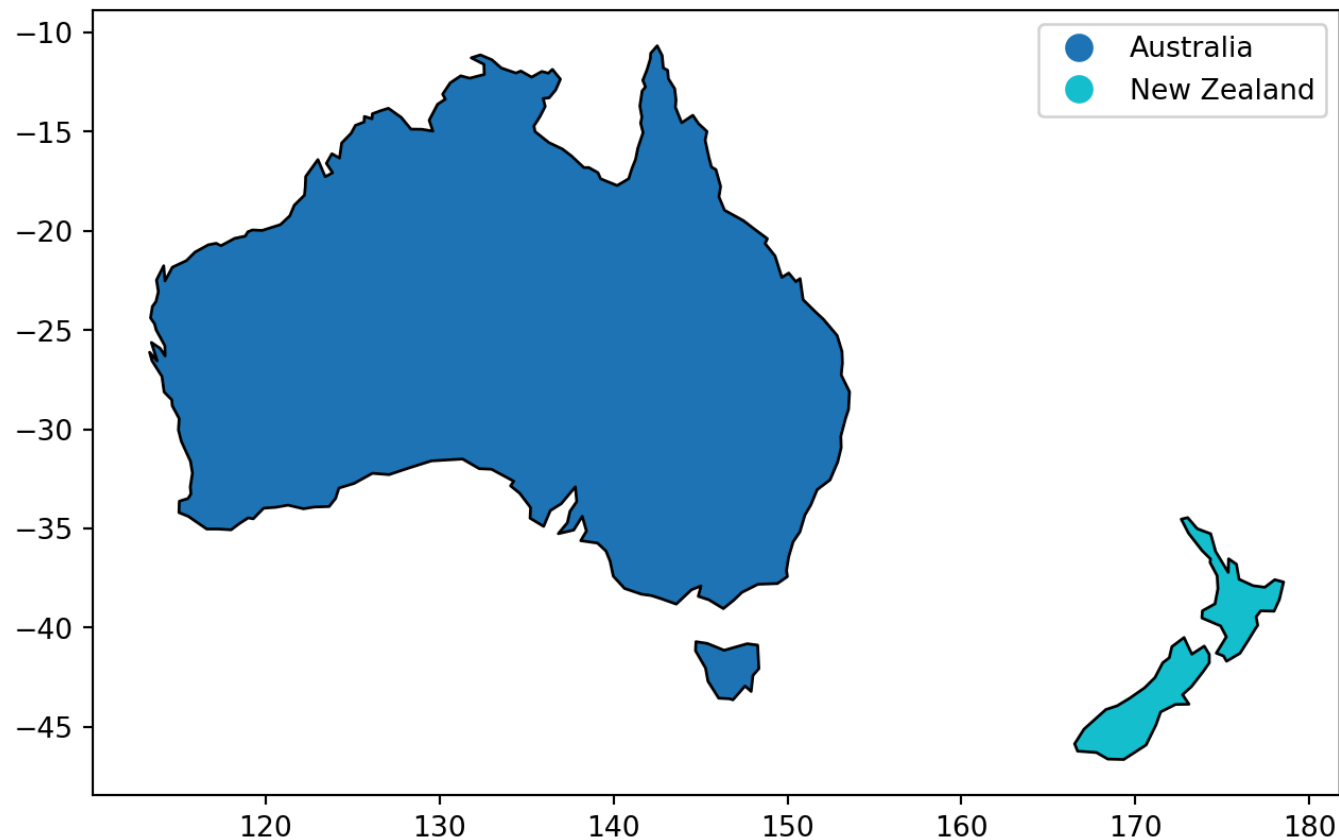
The World Map

```
1 fig, ax = plt.subplots()
2 df.plot(column = 'GDP_PC', cmap = 'Blues', edgecolor = 'lightgray', ax = ax,
3         legend = True, legend_kwds = {'orientation': 'horizontal', 'shrink': 0.6})
4 cities.plot(color = 'maroon', marker = 'o', markersize = 15, ax = ax)
5 for x, y, label in zip(cities.geometry.x, cities.geometry.y, cities['City']):
6     ax.annotate(label, xy = (x, y), xytext = (3, 3),
7                 textcoords = "offset points", fontsize = 6, color = 'maroon')
8 plt.show()
```



Australasia Map

```
1 countries = ['Australia', 'New Zealand']
2 df1 = df[df['SOVEREIGNTY'].isin(countries)]
3 fig, ax = plt.subplots()
4 df1.plot(column = 'SOVEREIGNTY', edgecolor = 'black', legend = True, ax = ax)
5 plt.show()
```



Singapore Map

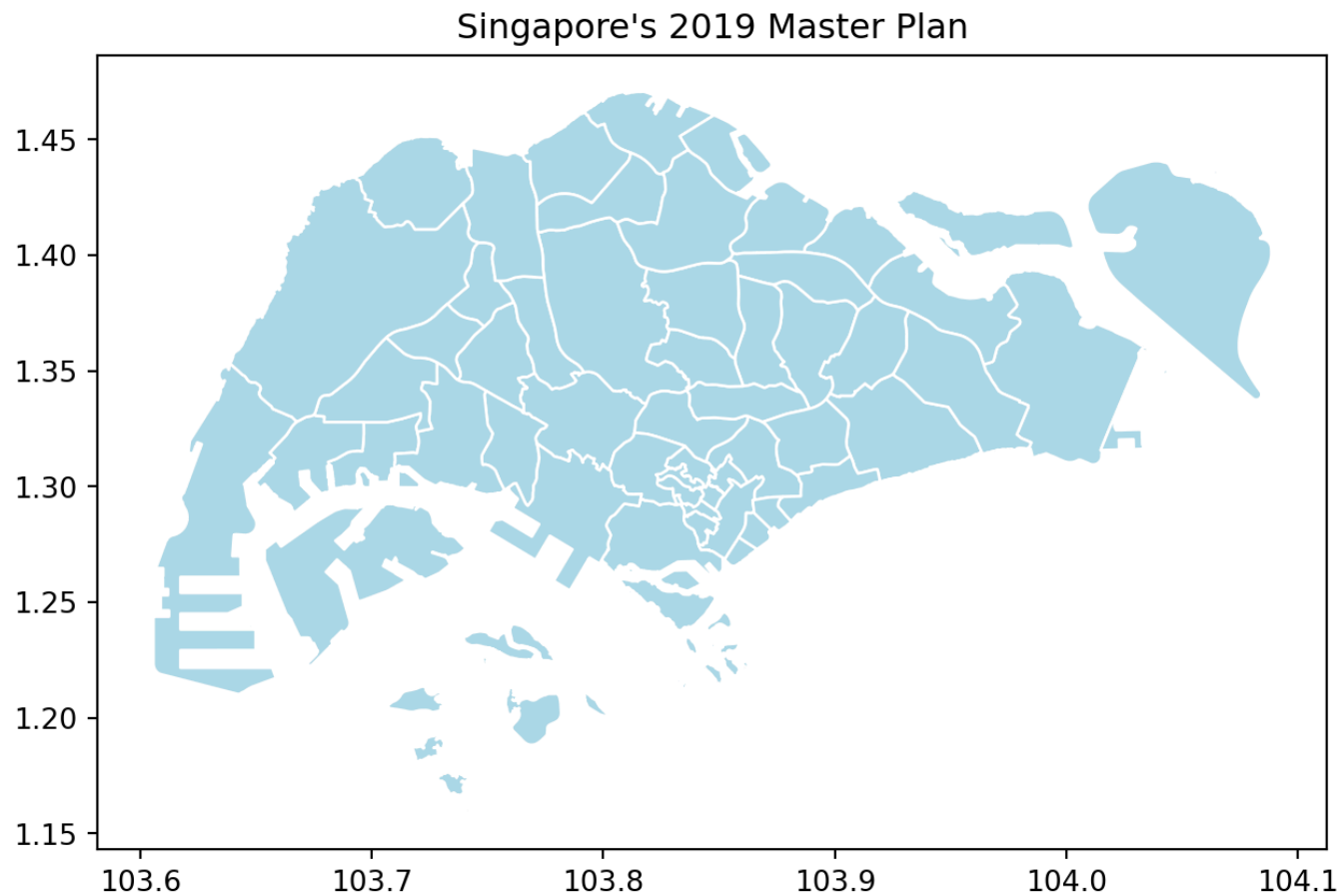
The file `wk12_MasterPlan2019.geojson` (from URA) contains Singapore's planning area boundary in 2019.

```
1 sg = gpd.read_file('../data/wk12_MasterPlan2019.geojson')
2 sg.head()
```

	town	geometry
0	BEDOK	MULTIPOLYGON (((103.93208 1.30555, 103.93208 1...
1	BOON LAY	MULTIPOLYGON (((103.72042 1.32824, 103.72003 1...
2	BUKIT BATOK	MULTIPOLYGON (((103.76408 1.37001, 103.76444 1...
3	BUKIT MERAH	MULTIPOLYGON (((103.82361 1.26018, 103.82362 1...
4	BUKIT PANJANG	MULTIPOLYGON (((103.77445 1.39029, 103.77499 1...

Singapore Map

```
1 fig, ax = plt.subplots()
2 sg.plot(color = 'lightblue', edgecolor = 'white', ax = ax)
3 ax.set_title("Singapore's 2019 Master Plan")
4 plt.show()
```



Singapore Map

- The file `wk12_sg_pop.csv` contains information about Singapore's population from 2021 to 2023.
- Let's extract population data in 2023.

```
1 sg_pop = pd.read_csv('../data/wk12_sg_pop.csv')
2 sg_pop = sg_pop[sg_pop['Time'] == 2023]
3 sg_pop.head()
```

	PA	Time	total	change
22	ANG MO KIO	2023	161910	10.0
45	BEDOK	2023	280230	1260.0
68	BISHAN	2023	88340	2400.0
91	BOON LAY	2023	0	0.0
114	BUKIT BATOK	2023	169360	4080.0

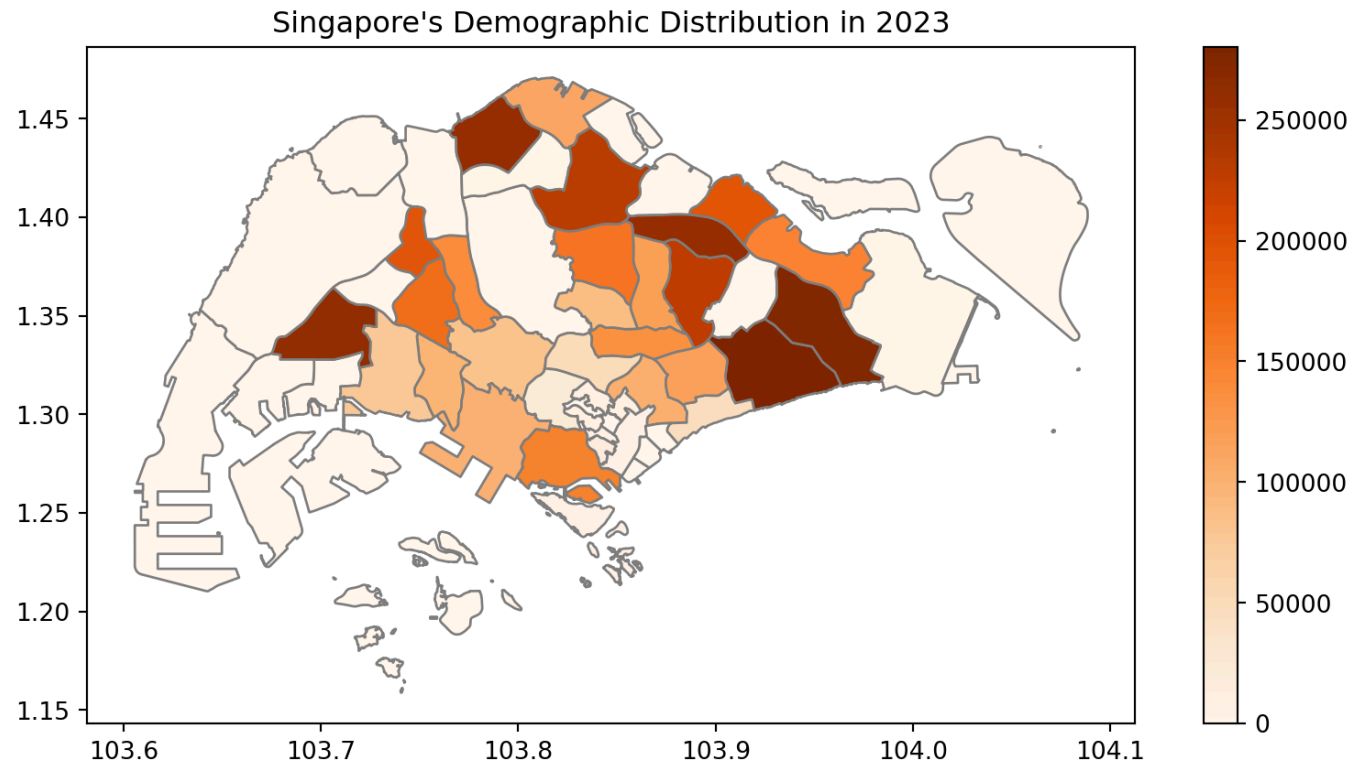
Singapore Map: Preparing Data

```
1 df = sg.merge(sg_pop, how = 'left', left_on = 'town', right_on = 'PA')
2 df.head()
```

	town	geometry	PA	Time	total	change
0	BEDOK	MULTIPOLYGON (((103.93208 1.30555, 103.93208 1...	BEDOK	2023	280230	1260.0
1	BOON LAY	MULTIPOLYGON (((103.72042 1.32824, 103.72003 1...	BOON LAY	2023	0	0.0
2	BUKIT BATOK	MULTIPOLYGON (((103.76408 1.37001, 103.76444 1...	BUKIT BATOK	2023	169360	4080.0
3	BUKIT MERAH	MULTIPOLYGON (((103.82361 1.26018, 103.82362 1...	BUKIT MERAH	2023	150840	210.0
4	BUKIT PANJANG	MULTIPOLYGON (((103.77445 1.39029, 103.77499 1...	BUKIT PANJANG	2023	139580	1850.0

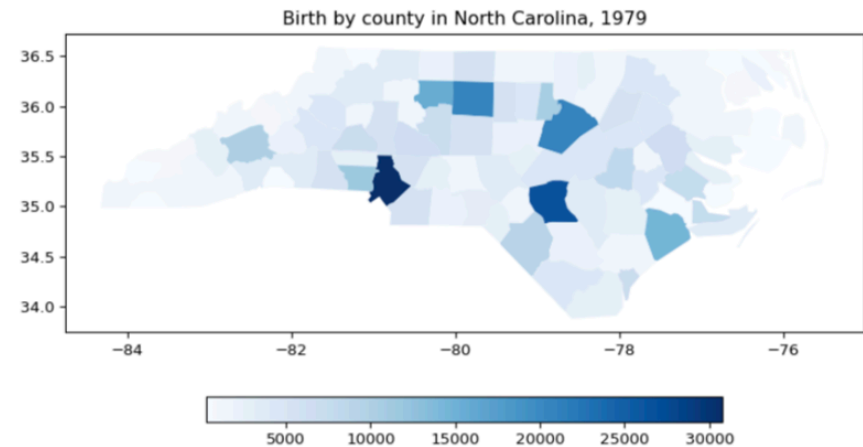
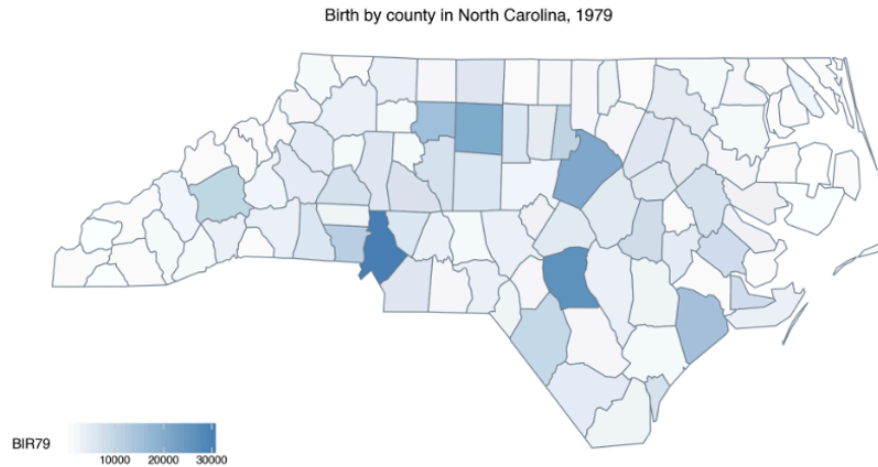
Singapore Map

```
1 fig, ax = plt.subplots()
2 df.plot(column = 'total', cmap = 'Oranges', edgecolor = 'gray',
3         legend = True, ax = ax)
4 ax.set_title("Singapore's Demographic Distribution in 2023")
5 plt.show()
```



Choropleth Map with `plotnine`

We have created choropleth maps with `plotnine` previously:

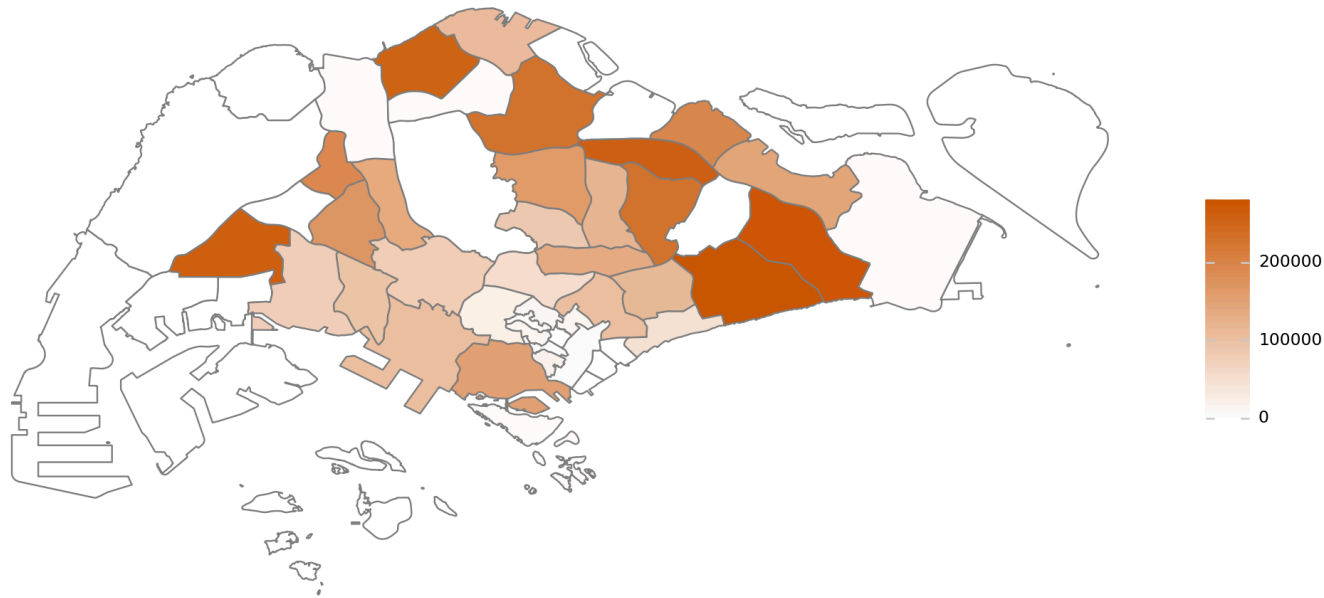


Let us now do the same for the Singapore map.

Choropleth Map with plotnine

```
1 from plotnine import *
2 (ggplot(df) +
3     geom_map(aes(fill = 'total'), color = 'gray') +
4     scale_fill_gradient(low = 'white', high = '#CC5500') +
5     labs(title = "Singapore's Demographic Distribution in 2023", fill = '') +
6     theme_void())
```

Singapore's Demographic Distribution in 2023



Summary

We have created choropleth maps with `GeoPandas` & `plotnine`.

- `GeoPandas` integrates naturally with `pandas`.
 - It is simple and efficient for exploratory mapping.
- `plotnine` offers more flexibility in styling and layering.
 - Better for publication-quality visuals.